AUTONOMOUS PRODUCTION CHOKE CONTROLLER

Problem Statement ID - Honeywell Hackathon, Problem 3A

Problem Statement Title - Autonomous Production Choke Controller for a Single Naturally Flowing Oil Well

Theme - Industrial Process Control / Oil & Gas Automation

PS Category - Software

Student Name - Divyansh Gupta (VIT Vellore)

PROCESS UNDERSTANDING & MODEL

Step test: 120 hr open-loop, choke 30->40->55->45->65 %, Ts = 1 hr

Response is first-order (time constant ~5 hr on oil rate), mild noise

Opening choke RAISES oil rate but LOWERS WHP / FLP / BHP

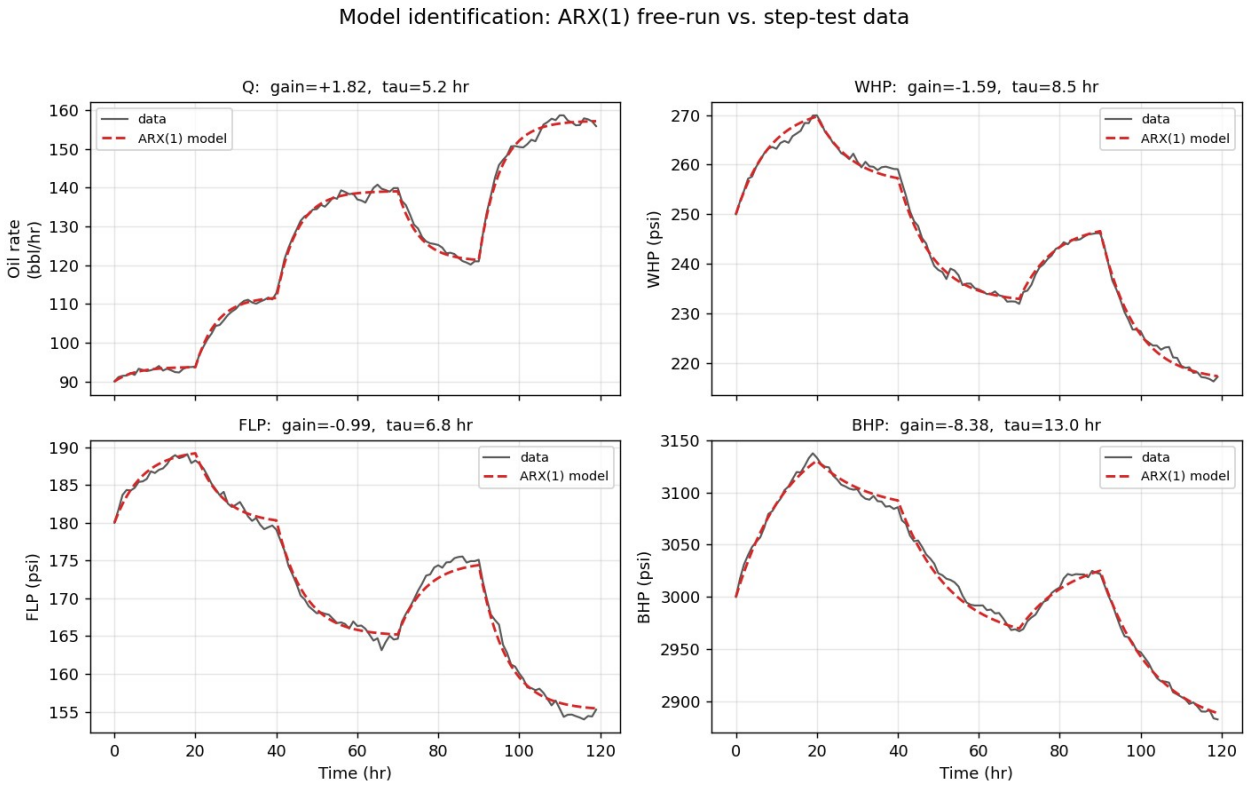
=> the binding safety constraints are LOWER pressure bounds

Model: ARX(1) per output by least squares

y[k+1] = a.y[k] + (1-a)(b0 + b1.u[k]); free-run fit 1.5-2.4 % of span

Identified steady-state gains (per % choke):

Q +1.82 bbl/hr WHP -1.59 FLP -0.99 BHP -8.38 psi



CONTROL STRATEGY

Simplified MPC by brute-force candidate search (per brief)

Each hour, enumerate choke +/- up to 5 % (ramp-feasible, fine grid)

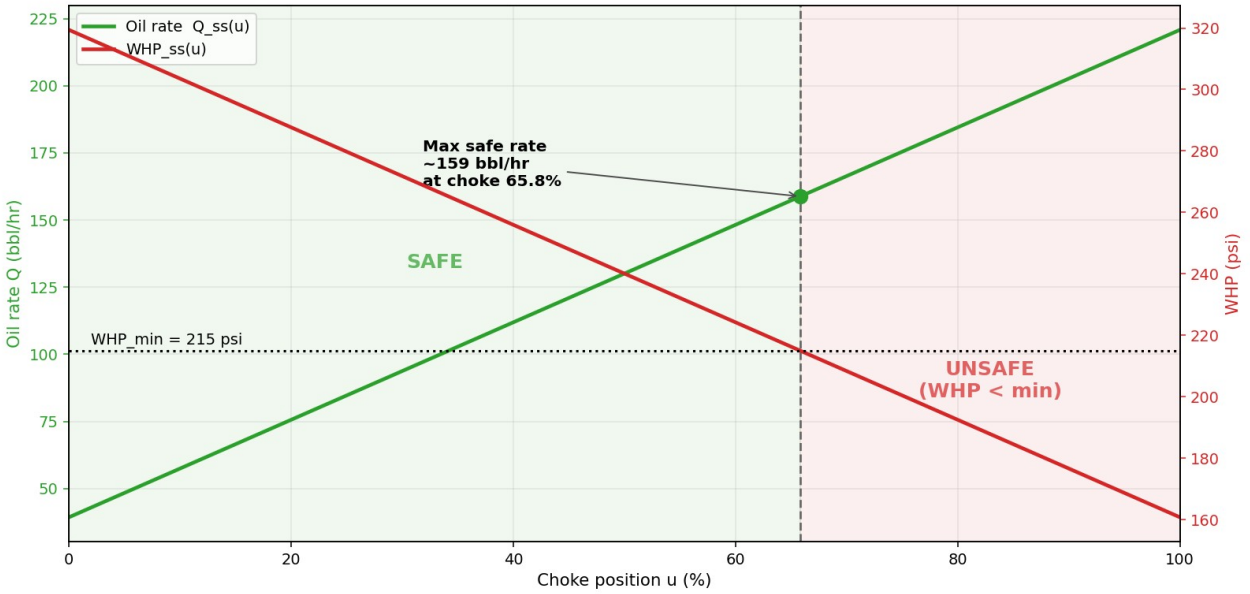
Predict each candidate's pressures over a settling horizon + steady state

REJECT any move predicted to breach WHP/FLP/BHP + safety margin

Among safe moves, minimise (Q_ss - target)^2 + lambda.(delta choke)^2

Infeasible target => same cost settles at the highest safe rate

Monotonic first-order response: steady-state feasibility proves the whole trajectory safe -> safe-by-construction, 0 violations



RESULTS - TARGET TRACKING (A & B)

Scenario A - Startup -> 130 bbl/hr

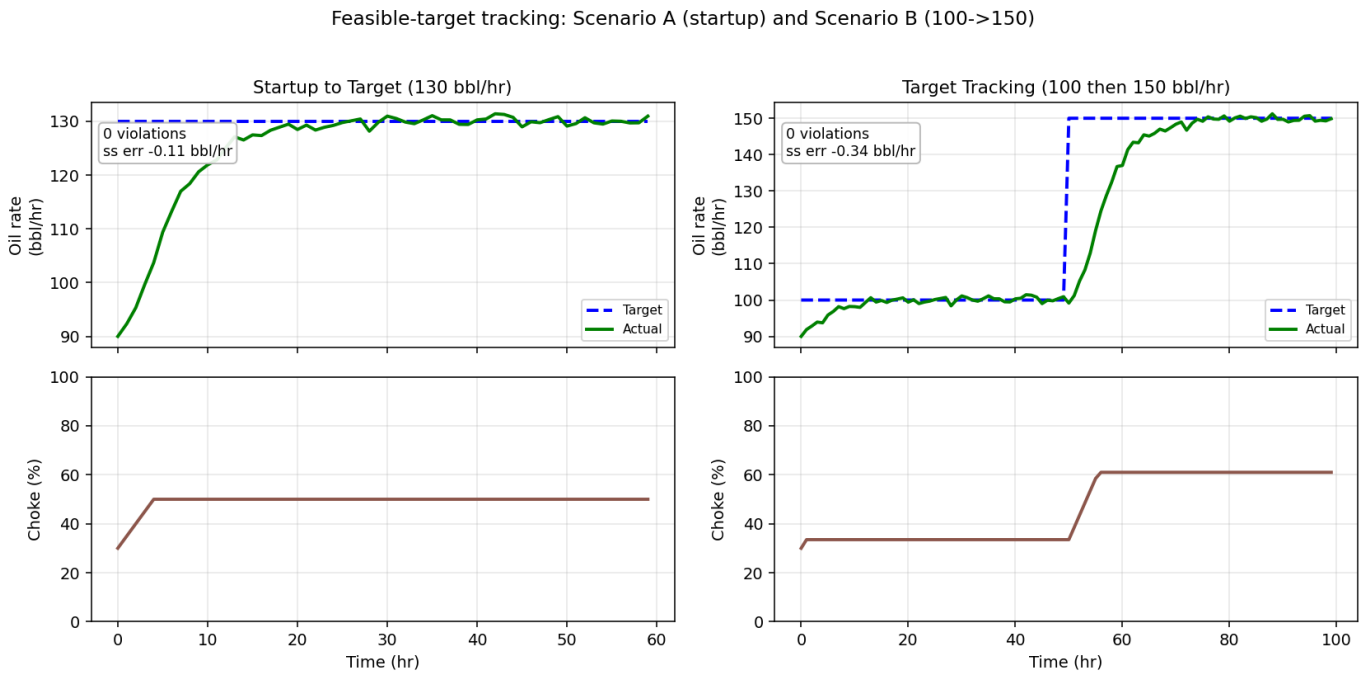
settles 129.9 bbl/hr, steady-state error -0.11, 0 violations

Scenario B - Target step 100 -> 150 bbl/hr

settles 149.7 bbl/hr, steady-state error -0.34, 0 violations

Pressures stay inside the envelope throughout

Choke moves are smooth and respect the +/-5 %/step ramp limit



SAFETY PERFORMANCE & LESSONS LEARNED

Scenario C - infeasible 185 bbl/hr: controller REFUSES,

settles at max safe 154.9 bbl/hr with 0 violations

Lessons learned:

Binding constraint is a lower pressure bound, not an upper rate cap

Monotonicity makes a cheap brute-force MPC provably safe

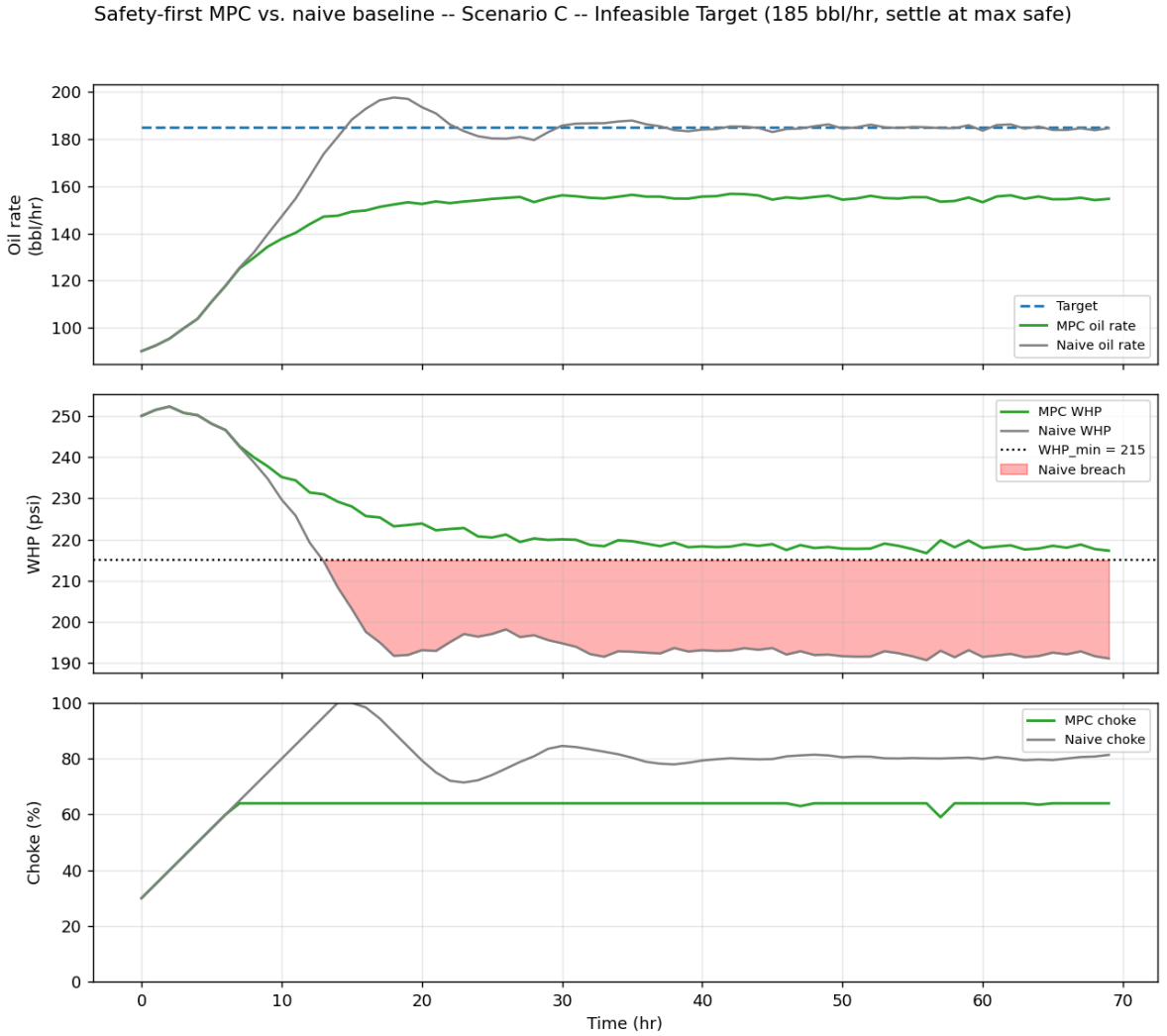
Safety margin trades ~4 bbl/hr for zero measured violations

0

MPC violations (A/B/C)

168

naive-baseline breaches on C



ARTIFACTS & REFERENCES

Code (flat Python modules at repo root):

dataio, model_id, envelope, simulator, controller, plotting, scenarios, main

Executed Jupyter notebook + technical report.md + results/ (plots, metrics.json)

Run: pip install -r requirements.txt && python main.py

main.py asserts 0 envelope violations across A/B/C

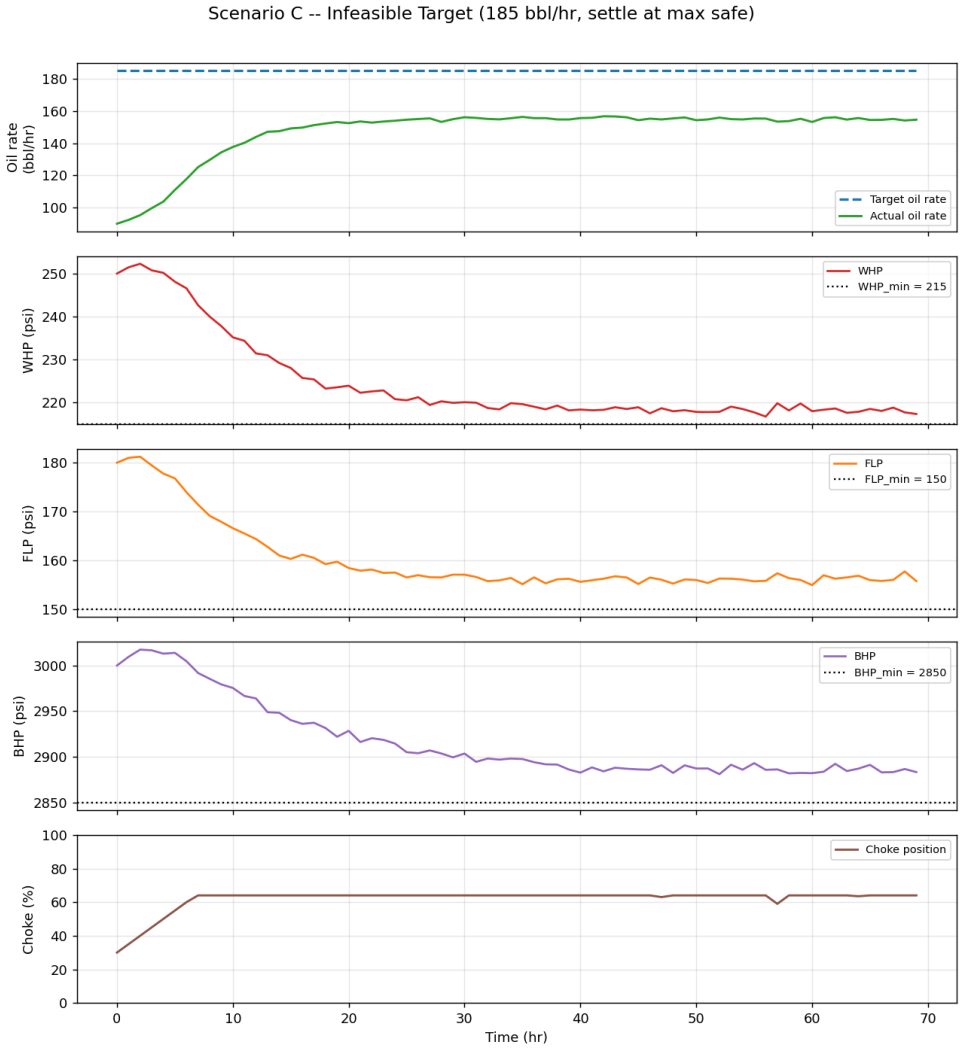
Simulator stand-in shares the official .step(choke)->(Q,WHP,FLP,BHP)

signature -> the real event simulator drops in with a one-line change

Repo: github.com/Divyansh2602/autonomous-choke-controller

References: least-squares system identification (Ljung); simplified

MPC by brute-force candidate evaluation (Honeywell brief)



Required 6-trend plot (Scenario C shown)

Technical Report — Autonomous Production Choke Controller (Problem 3A)

Honeywell Hackathon · Single naturally flowing oil well · Control interval Ts = 1 hr

---

1. Problem and approach

Set the production choke (0–100 %) once per hour to track a target oil rate while never violating the pressure operating envelope (lower bounds on wellhead, flowline and bottomhole pressure) or the ±5 %/step choke ramp limit. If a target is physically unsafe, line out at the maximum achievable safe rate rather than chase it.

Our solution is a **model-based predictive controller that is safe by construction**. We identify a first-order dynamic model of the well from the provided step test, then, every interval, search the reachable choke moves, predict each one's effect on all pressures, and **only ever command a move we have proven keeps the well inside the envelope**. The same decision rule tracks reachable targets tightly and refuses unreachable ones gracefully.

2. Data and open-loop analysis

The dataset is a 120-hour open-loop step test with the choke stepped 30 → 40 → 55 → 45 → 65 %. Two structural facts drive the whole design:

- The response is **first-order** with a time constant of roughly 5 hr on oil rate (slower on pressures), with mild measurement noise.
- **Opening the choke raises oil rate but lowers all three pressures.** The safety constraints are therefore **lower bounds**, and the well is pushed toward them precisely when we chase higher production — the core tension the controller must manage.

3. Dynamic model identification

For each output y ∈ {Q, WHP, FLP, BHP} we fit an ARX(1) model relating the choke *u* to the output one step ahead:

$$y[k+1] = a\, y[k] + (1-a)\,(b_0 + b_1\, u[k])$$

This is linear in the parameters, so we recover (a, b0, b1) by ordinary least squares (regress *y[k+1]* on *[y[k], 1, u[k]]*) and read off the physically meaningful quantities: steady-state gain $b_1 = dy_{cc}/du$, intercept b_0, and time constant $\tau = -Ts/\ln a$.

Output	gain b_1	b_0	τ (hr)	noise σ	free-run RMSE
Q (bbl/hr)	**+1.815**	39.31	5.15	0.74	1.5 % of span
WHP (psi)	**−1.586**	319.39	8.52	0.67	1.8 % of span
FLP (psi)	**−0.986**	219.27	6.76	0.52	2.4 % of span
BHP (psi)	**−8.383**	3417.35	12.96	2.95	1.9 % of span

The signs confirm the physics (choke up → rate up, pressures down). Validation is a genuine **free-run**: the model is initialised once and simulated open-loop under the recorded choke sequence using only its own predictions — never the measured output — and still matches the data to 1.5–2.4 % of signal span (`results/model_validation.png`). The one-step residual standard deviations set the noise level of the plant simulator so closed-loop tests see realistic scatter.

4. Operating envelope

The brief specifies no numeric envelope, so we adopt a **documented assumption**, chosen so the maximum safe rate sits high in the observed range and Scenario C is unambiguously infeasible. All limits live in one module (`envelope.py`):

Constraint	Value
WHP_min	215 psi
FLP_min	150 psi
BHP_min	2850 psi
Choke range	0–100 %
Ramp limit	±5 %/step

With the identified maps, **WHP is the first binding constraint**: it reaches 215 psi at choke ≈ 65.8 %,

giving a hard maximum safe rate of **≈ 159 bbl/hr**. When the official envelope/simulator is released, only this one file changes.

Per the brief, **wellhead temperature (WHT) and annulus pressure (AP)** are part of a complete production operating envelope but are *informational* here — not active constraints in this challenge — so they are not modelled or bounded. The envelope module is structured so they could be added as additional bounded outputs without touching the controller.

5. Controller design

A simplified MPC by brute-force candidate evaluation (explicitly permitted by the brief). Each interval:

1. **Read** current Q, WHP, FLP, BHP and choke position.
2. **Enumerate** candidate next positions: choke ± up to 5 %, on a 0.5 % grid, clipped to [0, 100] — every candidate already respects the ramp limit.
3. **Predict & screen for safety.** For each candidate, simulate the pressure trajectory over a settling horizon *and* compute its steady state with the ARX models. **Reject** the candidate if any pressure would fall below its bound plus a safety margin, at any predicted step or at steady state.
4. **Optimise.** Among the safe candidates, minimise `(Q_ss − target)² + λ·(Δchoke)²`.
5. **Infeasible target / recovery.** Because every safe candidate then has `Q_ss < target`, the cost automatically selects the **highest safe rate**. If no candidate is safe (e.g. the plant starts outside the envelope), the controller backs the choke off to the move that best restores the pressures.

Why this is safe by construction. A stable first-order response moves *monotonically* from the current output to its steady state. So if the current pressure is inside the envelope and the candidate's steady-state pressure is inside it (with margin), every intermediate value is too. Proving steady-state feasibility therefore proves trajectory feasibility — the controller never commits to a move it has not shown to be safe. The safety margin is sized to the fitted process noise (≈ 4σ on the binding pressure), which is what turns "predicted-safe" into "measured 0 violations" despite sensor scatter.

6. Results

Three required scenarios, run closed-loop against the simulator (`python main.py`, reproduced in `results/metrics.json`):

Scenario	Target	Settled rate	Settled choke	Steady-state error	Min WHP	Violations
A Startup → Target	130	129.9	50.0 %	−0.11 bbl/hr	238.8 psi	**0**
B Target Tracking	100 → 150	149.7	61.0 %	−0.34 bbl/hr	221.9 psi	**0**
C Infeasible	185	154.9	64.0 %	(infeasible)	216.7 psi	**0**

- **A** brings the well from startup (≈ 90 bbl/hr) to 130 and holds it, tracking to 0.11 bbl/hr with 18.8 psi of pressure headroom to spare.
- **B** tracks a mid-run target step 100 → 150, settling to 0.34 bbl/hr while WHP lines out at 221.9 psi — inside the envelope throughout.
- **C** requests 185 bbl/hr, which would require choke ≈ 80 % and drive WHP to ≈ 192 psi (well below the 215 floor). The controller **refuses**, settling at the maximum safe rate (154.9 bbl/hr) with WHP held at 216.7 psi and **zero violations**.

Each scenario produces the required six-trend plot (Target Q, Actual Q, WHP, FLP, BHP, Choke) in `results/scenario_{A,B,C}.png`.

7. Safety performance vs. a naive baseline

To quantify the value of the safety logic, the same scenarios were run with a naive proportional controller that chases the oil-rate target with no envelope awareness:

Scenario	Safety-first MPC violations	Naive baseline violations
A	0	0
B	0	0
C	**0**	**168** (WHP 57, FLP 56, BHP 55)

On the feasible targets (A, B) both reach the setpoint. On the infeasible target (C) the naive controller drives the choke past 100 %, holding the well ~65 psi below the WHP floor for the entire run — 168 constraint breaches — to gain just ~30 bbl/hr of unsafe, unsustainable production. The safety-first controller gives that up by design. `results/baseline_comparison_C.png` shows the contrast.

8. Lessons learned & limitations

- **The binding constraint is a lower pressure bound, not an upper rate bound** — recognising that inverted the intuition and made WHP the design driver.
- **Monotonicity is the key that makes a cheap MPC provably safe.** A short brute-force search plus a steady-state feasibility check is enough to guarantee the envelope; no heavy optimiser is required.
- **The safety margin trades a little production for a lot of robustness.** At ~4σ, measured violations are zero at the cost of settling ~4 bbl/hr below the theoretical hard limit — a deliberate, adjustable choice.
- **Limitations.** The plant model is first-order and linear. The brief's simulator assumptions (single naturally flowing well, no gas lift/ESP, constant reservoir properties, constant GOR and water cut) keep that valid for this challenge, but a real well would add mild nonlinearity, gas-coning and slugging the ARX model will not capture; the envelope numbers are also our assumption. The controller design is agnostic to all of this: `simulator.WellSimulator` shares the official `.step()` signature, so the real simulator drops in with a one-line change, and the envelope is a single editable module.

9. Reproducing

```
pip install -r requirements.txt
python main.py     # prints model params, runs A/B/C, asserts 0 violations
```

All figures and `metrics.json` are regenerated in `results/`. The narrative walk-through is in `notebook/choke_control_solution.ipynb`.

Appendix A - Result Figures

Figure 1 - ARX(1) model identification: free-run vs. step-test data

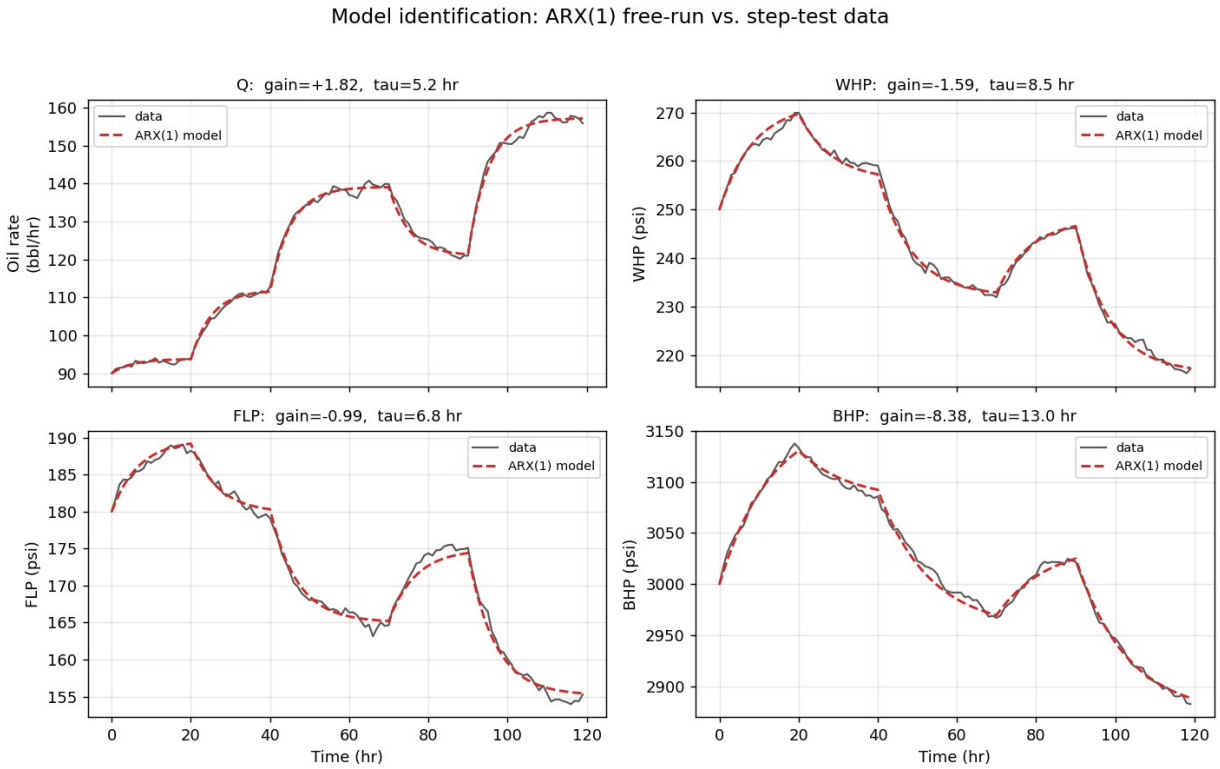


Figure 2 - Scenario A (Startup -> 130 bbl/hr): 6-trend plot

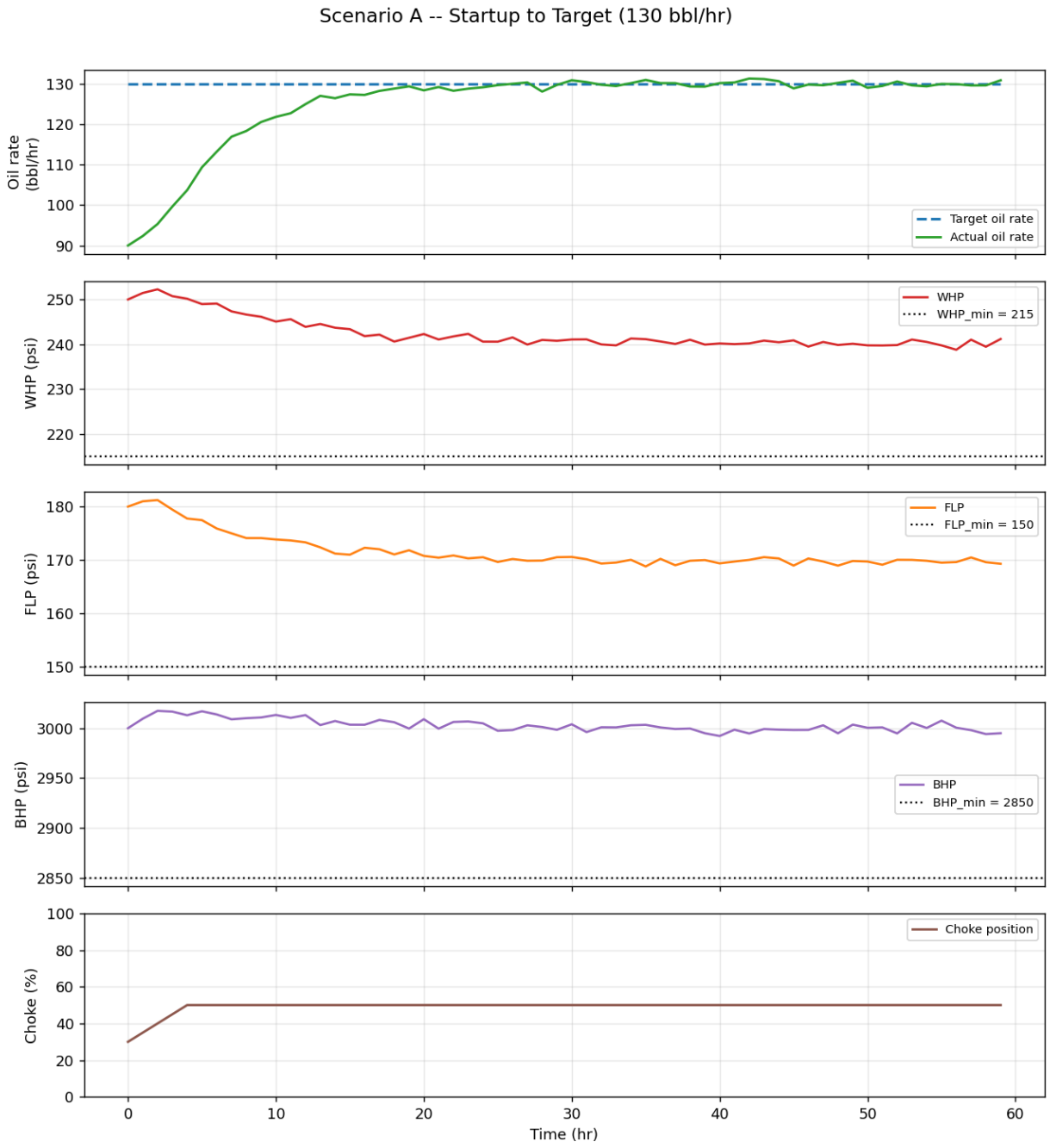


Figure 3 - Scenario B (Target tracking 100 -> 150 bbl/hr): 6-trend plot

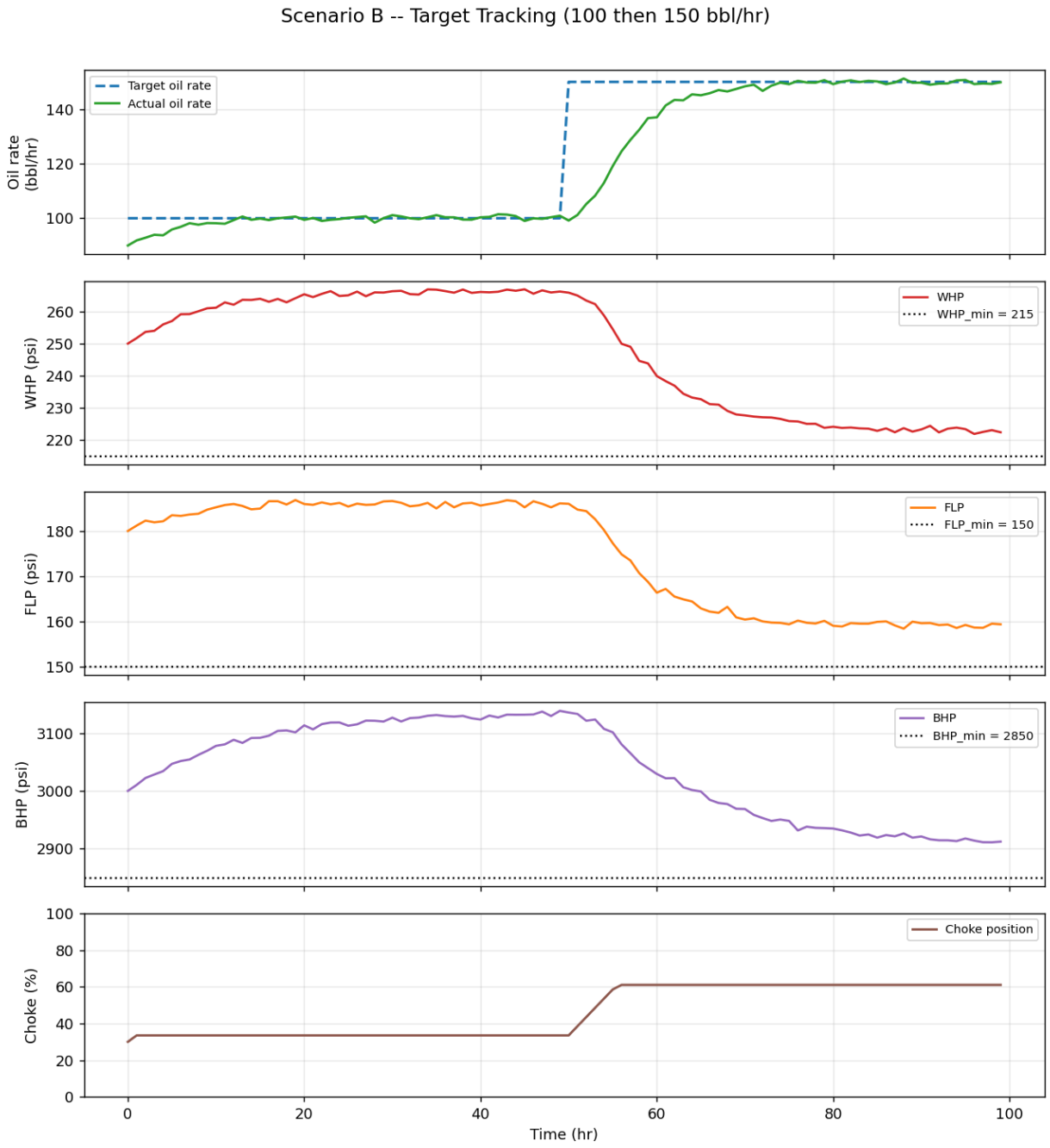


Figure 4 - Scenario C (Infeasible 185 bbl/hr, settle at max safe): 6-trend plot

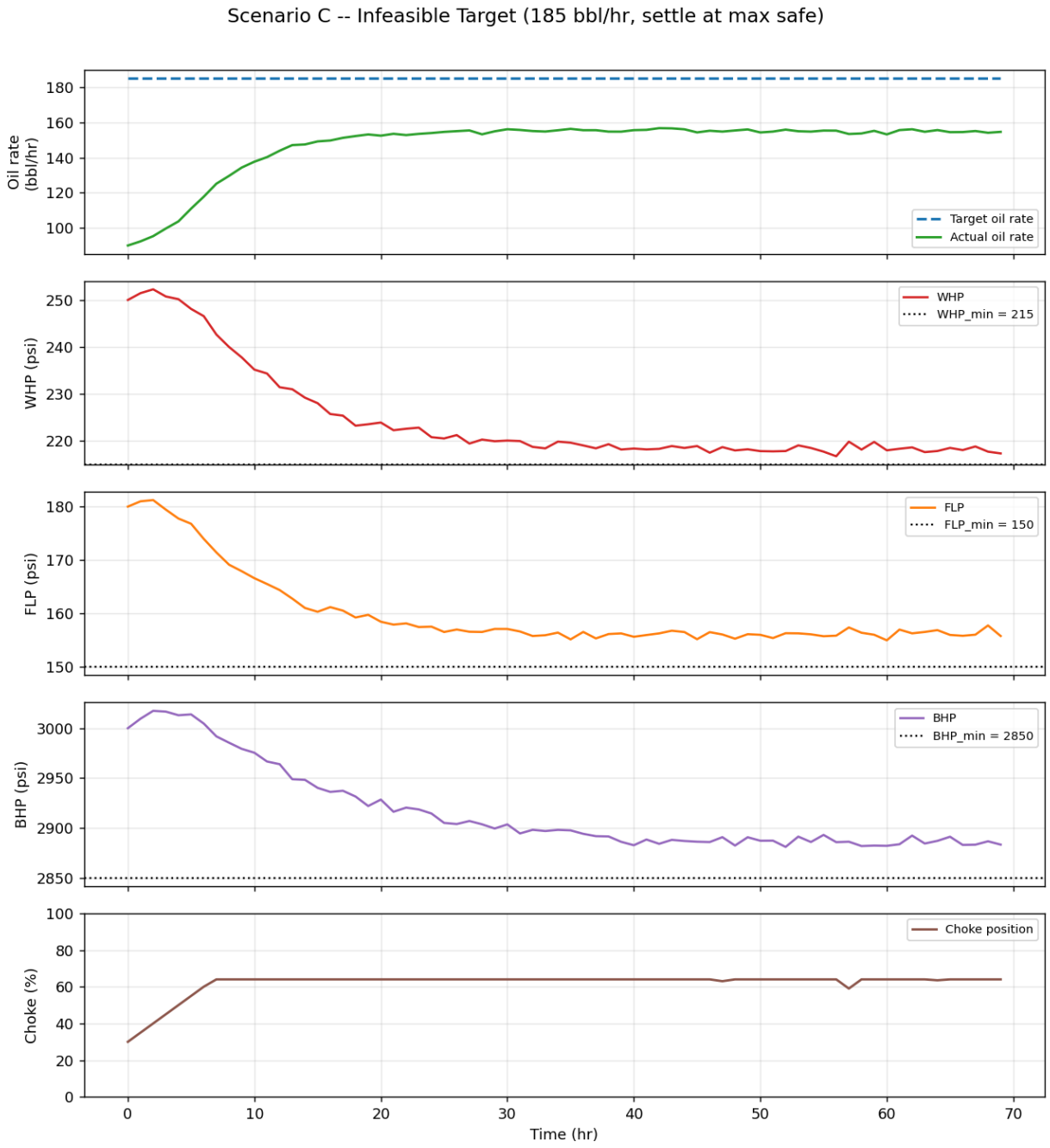


Figure 5 - Safety-first MPC vs. naive baseline on Scenario C

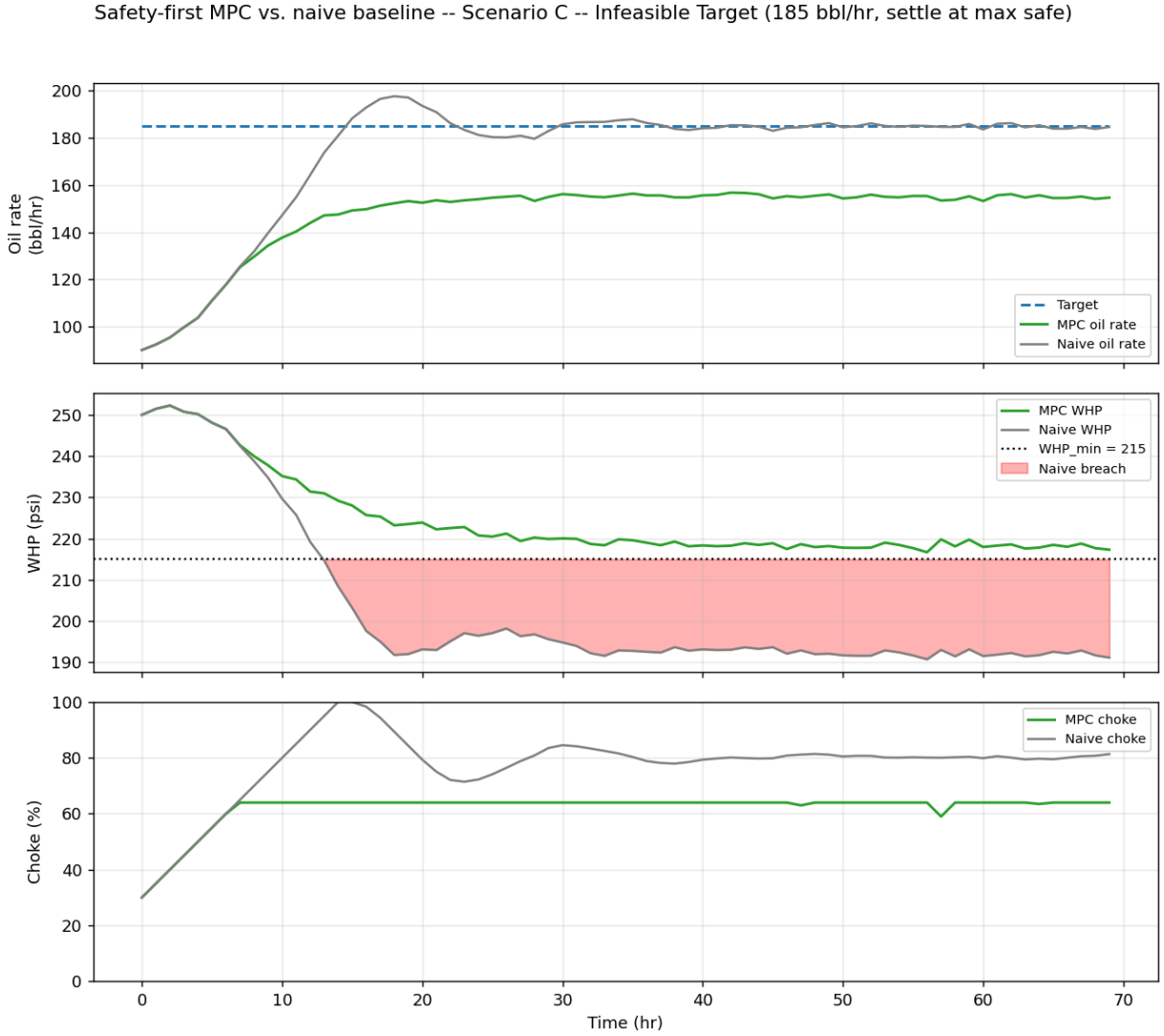
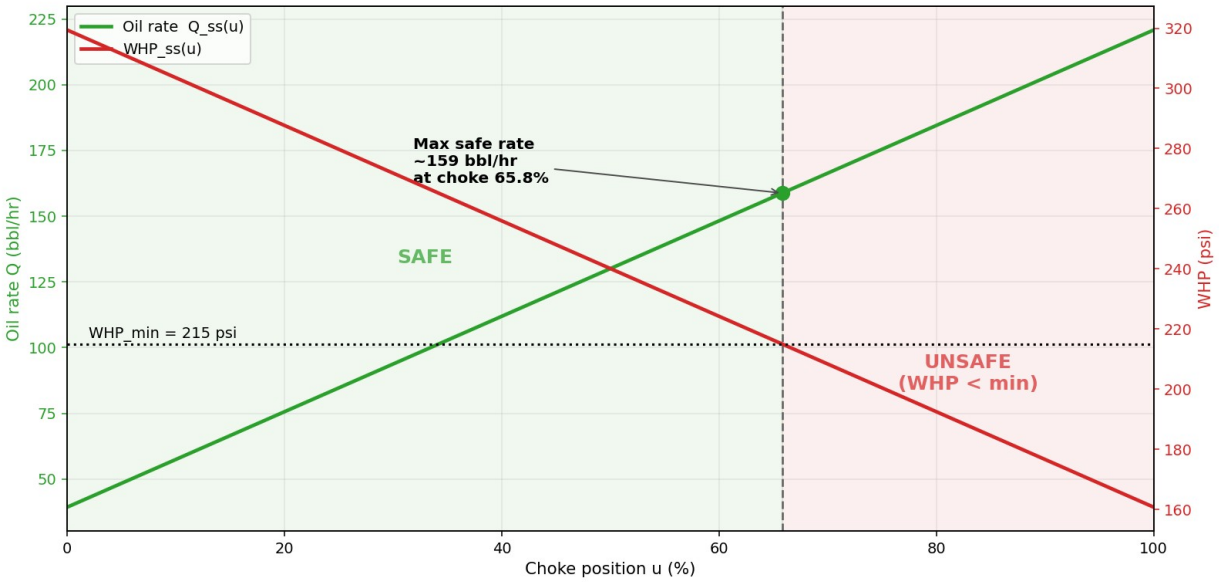


Figure 6 - Steady-state maps and the safe operating region



Appendix B - Source Code

dataio.py

```
"""Dataset loading and tidying for the choke-control step test.

The provided dataset is a 120-hour open-loop step test of a single naturally
flowing oil well. Control interval Ts = 1 hr. The raw columns are:

    Time_hr, Choke_pct, OilRate_bbl_hr, WHP_psi, FLP_psi, BHP_psi

Throughout the rest of the project we use short canonical names:

    t   -> Time_hr        (hr)
    u   -> Choke_pct      (%)
    Q   -> OilRate_bbl_hr (bbl/hr)
    WHP -> WHP_psi        (psi)   wellhead pressure
    FLP -> FLP_psi        (psi)   flowline pressure
    BHP -> BHP_psi        (psi)   bottomhole pressure
"""
from __future__ import annotations

from pathlib import Path

import pandas as pd

DATA_DIR = Path(__file__).resolve().parent / "data"
DEFAULT_CSV = DATA_DIR / "Autonomous_Choke_Control_Simulated_Dataset.csv"
```

```
# Raw dataset column names, in the order we rely on.

RAW_COLUMNS = [

    "Time_hr",

    "Choke_pct",

    "OilRate_bbl_hr",

    "WHP_psi",

    "FLP_psi",

    "BHP_psi",

]



# Map raw column -> canonical short name used across the codebase.

CANONICAL = {

    "Time_hr": "t",

    "Choke_pct": "u",

    "OilRate_bbl_hr": "Q",

    "WHP_psi": "WHP",

    "FLP_psi": "FLP",

    "BHP_psi": "BHP",

}



# The four measured process outputs the controller reasons about.

OUTPUTS = ["Q", "WHP", "FLP", "BHP"]





def load_step_test(path: str | Path = DEFAULT_CSV) -> pd.DataFrame:

    """Load the step-test CSV, validate it, and return a tidy frame.
```

```
    The returned frame keeps the raw column names, is sorted by time, and is
    guaranteed to contain every required column with no missing values. We
    fail loudly on any structural problem rather than silently returning a
    partial frame that would corrupt the identification downstream.
    """
    path = Path(path)
    if not path.exists():
        raise FileNotFoundError(
            f"Step-test dataset not found at {path}. It should be committed at "
            f"{DEFAULT_CSV.relative_to(DATA_DIR.parent)}."
        )

    df = pd.read_csv(path)

    missing = [c for c in RAW_COLUMNS if c not in df.columns]
    if missing:
        raise ValueError(
            f"Dataset {path.name} is missing required columns: {missing}. "
            f"Found columns: {list(df.columns)}."
        )

    df = df[RAW_COLUMNS].copy()

    if df.isnull().any().any():
        bad = df.columns[df.isnull().any()].tolist()
```

```
        raise ValueError(f"Dataset {path.name} has missing values in columns: {bad}.")



    df = df.sort_values("Time_hr").reset_index(drop=True)

    return df




def to_canonical(df: pd.DataFrame) -> pd.DataFrame:

    """Return a copy with canonical short column names (t, u, Q, WHP, FLP, BHP)."""

    return df.rename(columns=CANONICAL)




def infer_sample_time(df: pd.DataFrame) -> float:

    """Infer the sampling interval (hr) from the Time_hr column.



    Raises if the sampling is not uniform, because the ARX(1) identification and

    the time-constant conversion both assume a fixed Ts.

    """

    dt = df["Time_hr"].diff().dropna().to_numpy()

    if len(dt) == 0:

        raise ValueError("Cannot infer sample time from a single-row dataset.")

    ts = float(dt[0])

    if not (dt == ts).all():

        raise ValueError(

            f"Non-uniform sampling detected in Time_hr (steps: {sorted(set(dt))}). "

            "ARX(1) identification assumes a fixed sample interval."

        )
```

```
    return ts




if __name__ == "__main__":

    frame = load_step_test()

    print(f"Loaded {len(frame)} rows from {DEFAULT_CSV.name}")

    print(f"Inferred Ts = {infer_sample_time(frame)} hr")

    print("\nChoke schedule (step changes):")

    steps = frame[frame["Choke_pct"].diff().fillna(1) != 0]

    for _, row in steps.iterrows():

        print(f"  t={int(row['Time_hr']):>3} hr  ->  choke {row['Choke_pct']:.0f}%")

    print("\nHead:")

    print(frame.head().to_string(index=False))
```

model_id.py

```
"""ARX(1) dynamic model identification from the open-loop step test.



For each output y in {Q, WHP, FLP, BHP} we fit a first-order ARX model that

maps the choke position u to the output one step ahead:



    y[k+1] = a * y[k] + (1 - a) * (b0 + b1 * u[k])



This is linear in the parameters, so we regress y[k+1] on [y[k], 1, u[k]] by

ordinary least squares to recover (a, c0, c1), then map back to the physically

meaningful form:



    b1 = c1 / (1 - a)        steady-state gain     dy_ss/du

    b0 = c0 / (1 - a)        steady-state intercept

    tau = -Ts / ln(a)        first-order time constant (hr)



The one-step residual standard deviation gives the process noise level, which

the simulator reuses so its measurements have realistic scatter.

"""

from __future__ import annotations



from dataclasses import dataclass



import numpy as np

import pandas as pd
```

```
from dataio import OUTPUTS, to_canonical




@dataclass(frozen=True)

class ARXModel:

    """Identified first-order ARX(1) model for one output.



    ``y[k+1] = a * y[k] + (1 - a) * (b0 + b1 * u[k])``

    """



    name: str

    a: float          # pole (0 < a < 1 for a stable first-order response)

    b0: float         # steady-state intercept

    b1: float         # steady-state gain dy_ss/du

    resid_std: float  # one-step-ahead residual std (process noise level)

    Ts_hr: float      # sample interval used for the time-constant conversion



    @property

    def gain(self) -> float:

        """Steady-state gain dy_ss/du (== b1)."""

        return self.b1



    @property

    def tau_hr(self) -> float:

        """First-order time constant (hr). Infinite if a >= 1 (non-decaying)."""

        if not (0.0 < self.a < 1.0):
```

```
            return float("inf")

        return -self.Ts_hr / np.log(self.a)



    def predict_next(self, y: float, u: float) -> float:

        """One-step-ahead prediction y[k+1] from current output y and input u."""

        return self.a * y + (1.0 - self.a) * (self.b0 + self.b1 * u)



    def steady_state(self, u: float) -> float:

        """Steady-state output the model settles to when the choke is held at u."""

        return self.b0 + self.b1 * u




def identify_arx(y: np.ndarray, u: np.ndarray, name: str, Ts: float = 1.0) -> ARXModel:

    """Identify a single ARX(1) model by OLS.



    Parameters

    ----------

    y : array of output samples y[0..N-1].

    u : array of input samples  u[0..N-1] (choke %).

    name : output label.

    Ts : sample interval (hr).

    """

    y = np.asarray(y, dtype=float)

    u = np.asarray(u, dtype=float)

    if y.shape != u.shape or y.ndim != 1:

        raise ValueError("y and u must be 1-D arrays of equal length.")
```

```
    if len(y) < 4:

        raise ValueError(f"Need at least 4 samples to identify '{name}', got {len(y)}.")



    # Regress y[k+1] on [y[k], 1, u[k]].

    y_k = y[:-1]

    y_k1 = y[1:]

    u_k = u[:-1]

    X = np.column_stack([y_k, np.ones_like(y_k), u_k])



    theta, *_ = np.linalg.lstsq(X, y_k1, rcond=None)

    a, c0, c1 = (float(v) for v in theta)



    if a >= 1.0:

        raise ValueError(

            f"Identified pole a={a:.4f} >= 1 for '{name}': model is not a stable "

            "first-order response; check the data."

        )



    b1 = c1 / (1.0 - a)

    b0 = c0 / (1.0 - a)



    resid = y_k1 - X @ theta

    # dof correction: N-1 equations minus 3 fitted parameters.

    dof = max(1, len(resid) - 3)

    resid_std = float(np.sqrt(np.sum(resid**2) / dof))
```

```
    return ARXModel(name=name, a=a, b0=b0, b1=b1, resid_std=resid_std, Ts_hr=Ts)




def identify_all(df: pd.DataFrame, Ts: float = 1.0) -> dict[str, ARXModel]:

    """Identify an ARX(1) model for every output in a tidy (canonical) frame.



    Accepts either raw or canonical column names.

    """

    canon = df if "u" in df.columns else to_canonical(df)

    u = canon["u"].to_numpy(dtype=float)

    return {name: identify_arx(canon[name].to_numpy(float), u, name, Ts) for name in OUTPUTS}




def free_run(model: ARXModel, u: np.ndarray, y0: float) -> np.ndarray:

    """Simulate the model open-loop (free-run) from y0 under an input sequence.



    Uses only the model's own predictions (never the measured output), so the

    result is a genuine multi-step validation, not a one-step-ahead cheat.

    """

    u = np.asarray(u, dtype=float)

    y = np.empty(len(u), dtype=float)

    y[0] = y0

    for k in range(len(u) - 1):

        y[k + 1] = model.predict_next(y[k], u[k])

    return y
```

```
def validation_fit(model: ARXModel, df: pd.DataFrame) -> dict[str, float]:

    """Free-run the model over the dataset and report fit quality vs the data."""

    canon = df if "u" in df.columns else to_canonical(df)

    y_meas = canon[model.name].to_numpy(float)

    u = canon["u"].to_numpy(float)

    y_sim = free_run(model, u, y0=y_meas[0])



    err = y_sim - y_meas

    rmse = float(np.sqrt(np.mean(err**2)))

    span = float(np.ptp(y_meas)) or 1.0

    return {

        "rmse": rmse,

        "rmse_pct_of_span": 100.0 * rmse / span,

        "max_abs_err": float(np.max(np.abs(err))),

    }





def format_report(models: dict[str, ARXModel]) -> str:

    """Human-readable summary of the identified parameters."""

    lines = [

        "Identified ARX(1) models:  y[k+1] = a*y[k] + (1-a)*(b0 + b1*u[k])",

        "-" * 68,

        f"{'output':<6}{'a':>8}{'gain b1':>12}{'b0':>12}{'tau (hr)':>11}{'noise sd':>11}",

        "-" * 68,

    ]
```

```
    for name in OUTPUTS:
        m = models[name]
        lines.append(
            f"{m.name:<6}{m.a:>8.4f}{m.b1:>12.4f}{m.b0:>12.3f}"
            f"{m.tau_hr:>11.2f}{m.resid_std:>11.3f}"
        )
    lines.append("-" * 68)
    return "\n".join(lines)




if __name__ == "__main__":
    from dataio import infer_sample_time, load_step_test

    frame = load_step_test()
    ts = infer_sample_time(frame)
    mdls = identify_all(frame, Ts=ts)
    print(format_report(mdls))
    print("\nFree-run validation (model simulated open-loop vs data):")
    for out_name in OUTPUTS:
        v = validation_fit(mdls[out_name], frame)
        print(
            f"  {out_name:<4} RMSE={v['rmse']:.3f} "
            f"({v['rmse_pct_of_span']:.1f}% of span, max abs err {v['max_abs_err']:.2f})"
        )
```

envelope.py

```
"""Operating envelope and actuator limits -- the single source of truth.



The hackathon brief does not give explicit envelope numbers, so these are a

DOCUMENTED ASSUMPTION (see HANDOFF.md Section 3). They are chosen so that the

maximum safe production rate sits high in the observed range and Scenario C is

clearly infeasible, with WHP as the first binding constraint.



When the official envelope / simulator is announced at the event, edit ONLY this

file -- every other module reads its limits from here.

"""

from __future__ import annotations



from dataclasses import dataclass





# The pressure outputs that carry a safety constraint. All three are LOWER

# bounds: opening the choke raises oil rate but pulls every pressure down, so

# the binding constraints are floors, not ceilings (see HANDOFF.md Section 2).

PRESSURE_OUTPUTS = ("WHP", "FLP", "BHP")





@dataclass(frozen=True)

class Envelope:

    """Immutable container for all operating constraints.
```

```
    Attributes
    ----------
    WHP_min, FLP_min, BHP_min : float
        Lower pressure bounds (psi). Violating any of these is a hard safety
        breach.
    choke_min, choke_max : float
        Absolute choke position limits (%).
    max_ramp : float
        Maximum change in choke position per control interval (% per step).
    Ts_hr : float
        Control interval (hr).
    """

    WHP_min: float = 215.0
    FLP_min: float = 150.0
    BHP_min: float = 2850.0
    choke_min: float = 0.0
    choke_max: float = 100.0
    max_ramp: float = 5.0
    Ts_hr: float = 1.0

    def pressure_mins(self) -> dict[str, float]:
        """Return the lower bound for each constrained pressure output."""
        return {"WHP": self.WHP_min, "FLP": self.FLP_min, "BHP": self.BHP_min}

    def clip_choke(self, u: float) -> float:
```

```
        """Clip a choke position to the absolute [choke_min, choke_max] range."""
        return float(min(self.choke_max, max(self.choke_min, u)))

    def apply_ramp(self, u_target: float, u_current: float) -> float:
        """Clamp a desired choke move to the ramp limit, then to absolute bounds.

        Guarantees ``|return - u_current| <= max_ramp`` and the result lies in
        ``[choke_min, choke_max]``.
        """
        delta = u_target - u_current
        if delta > self.max_ramp:
            delta = self.max_ramp
        elif delta < -self.max_ramp:
            delta = -self.max_ramp
        return self.clip_choke(u_current + delta)

    def pressure_violations(self, whp: float, flp: float, bhp: float) -> dict[str, bool]:
        """Return, per pressure, whether it is strictly below its lower bound."""
        return {
            "WHP": whp < self.WHP_min,
            "FLP": flp < self.FLP_min,
            "BHP": bhp < self.BHP_min,
        }

    def any_violation(self, whp: float, flp: float, bhp: float) -> bool:
        """True if any constrained pressure is below its lower bound."""
```

```
        return any(self.pressure_violations(whp, flp, bhp).values())



    def worst_margin(self, whp: float, flp: float, bhp: float) -> float:

        """Smallest (pressure - lower_bound) across the three pressures (psi).



        Positive means all pressures are inside the envelope; the value is the

        tightest remaining headroom. Negative means at least one bound is

        breached, by that amount. Used as the fallback objective when no

        candidate move is safe -- maximise the worst margin to climb back in.

        """

        return min(

            whp - self.WHP_min,

            flp - self.FLP_min,

            bhp - self.BHP_min,

        )





# Default envelope instance used across the project.

DEFAULT_ENVELOPE = Envelope()







if __name__ == "__main__":

    env = DEFAULT_ENVELOPE

    print("Operating envelope (single source of truth):")

    print(f"  WHP_min = {env.WHP_min} psi")

    print(f"  FLP_min = {env.FLP_min} psi")
```

```
    print(f"  BHP_min = {env.BHP_min} psi")

    print(f"  choke   in [{env.choke_min}, {env.choke_max}] %")

    print(f"  ramp    <= {env.max_ramp} % per {env.Ts_hr} hr step")
```

simulator.py

```
"""WellSimulator -- a plant stand-in for closed-loop testing.



This is a drop-in surrogate for the official event simulator. It exposes the

exact same interface the brief specifies:



    Q, WHP, FLP, BHP = simulator.step(choke_position)



so swapping in the real simulator is a one-line change in the scenario runner.



Internally it advances the four identified ARX(1) models by one control

interval and adds measurement noise at the level fitted from the step-test

residuals. It clips the commanded choke to the physical [0, 100] range (the

plant cannot exceed that); the ramp limit is an actuator/controller constraint

and is intentionally NOT enforced here, so the simulator faithfully reports what

would happen if a controller ever commanded an aggressive move.

"""

from __future__ import annotations



import numpy as np



from dataio import OUTPUTS

from model_id import ARXModel





class WellSimulator:
```

```
    """First-order well model with the official simulator's ``step`` signature."""



    def __init__(

        self,

        models: dict[str, ARXModel],

        u0: float,

        y0: dict[str, float] | None = None,

        noise: bool = True,

        seed: int | None = None,

    ) -> None:

        """

        Parameters

        ----------

        models : identified ARX(1) model per output (Q, WHP, FLP, BHP).

        u0 : initial choke position (%).

        y0 : optional explicit initial outputs. If omitted, each output starts

            at the model's steady state for u0 (a well already lined out at u0).

        noise : add fitted measurement noise to each reported output.

        seed : RNG seed for reproducibility.

        """

        missing = [o for o in OUTPUTS if o not in models]

        if missing:

            raise ValueError(f"WellSimulator missing models for outputs: {missing}")



        self.models = models

        self.noise = noise
```

```
        self._rng = np.random.default_rng(seed)



        self.u = float(u0)

        if y0 is None:

            self.state = {o: models[o].steady_state(u0) for o in OUTPUTS}

        else:

            missing_y = [o for o in OUTPUTS if o not in y0]

            if missing_y:

                raise ValueError(f"y0 is missing initial values for: {missing_y}")

            self.state = {o: float(y0[o]) for o in OUTPUTS}



    def measure(self) -> tuple[float, float, float, float]:

        """Return the current (noise-free) outputs as (Q, WHP, FLP, BHP)."""

        return tuple(self.state[o] for o in OUTPUTS)  # type: ignore[return-value]



    def step(self, choke_position: float) -> tuple[float, float, float, float]:

        """Apply a choke position for one interval; return (Q, WHP, FLP, BHP).



        The commanded choke is clipped to the physical [0, 100] range. The

        internal (true) state advances noise-free; the returned measurement has

        fitted noise added on top, mirroring a real sensor reading.

        """

        u = float(min(100.0, max(0.0, choke_position)))

        self.u = u



        reported: list[float] = []
```

```
        for o in OUTPUTS:
            m = self.models[o]
            self.state[o] = m.predict_next(self.state[o], u)
            value = self.state[o]
            if self.noise and m.resid_std > 0:
                value = value + self._rng.normal(0.0, m.resid_std)
            reported.append(float(value))

        return tuple(reported)  # type: ignore[return-value]


# Startup state used by the demonstration scenarios: the well lined out at the
# low choke setting from the top of the step-test dataset (choke 30%, Q ~ 90).
STARTUP_STATE = {"Q": 90.0, "WHP": 250.0, "FLP": 180.0, "BHP": 3000.0}
STARTUP_CHOKE = 30.0


if __name__ == "__main__":
    from model_id import identify_all
    from dataio import load_step_test

    mdls = identify_all(load_step_test())
    sim = WellSimulator(mdls, u0=STARTUP_CHOKE, y0=STARTUP_STATE, noise=True, seed=0)
    print("Step response to a fixed 50% choke from startup:")
    print(f"{'k':>3}{'choke':>7}{'Q':>9}{'WHP':>9}{'FLP':>9}{'BHP':>10}")
    for k in range(0, 16):
```

```
        q, whp, flp, bhp = sim.step(50.0)

        if k % 3 == 0:

            print(f"{k:>3}{50.0:>7.1f}{q:>9.2f}{whp:>9.2f}{flp:>9.2f}{bhp:>10.2f}")
```

controller.py

```
"""ChokeMPC -- a simplified, safety-first predictive choke controller.



The brief explicitly permits a brute-force / candidate-search MPC. Each control

interval the controller:



  1. Reads the current measurements (Q, WHP, FLP, BHP) and choke position.

  2. Enumerates candidate next positions within the ramp limit

     (choke +/- up to max_ramp, on a fine grid), clipped to [0, 100].

  3. For each candidate, predicts the pressure trajectory over a settling

     horizon with the internal ARX(1) models AND the eventual steady state, and

     REJECTS the candidate if any pressure would fall below its lower bound plus

     a safety margin -- at any predicted step or at steady state.

  4. Among the safe (feasible) candidates, minimises

         cost = (Q_ss(candidate) - target)^2 + lambda * (delta_choke)^2

     which tracks the target when it is reachable and, because every safe

     candidate then has Q_ss < target, automatically settles at the highest

     safe rate when the target is infeasible.

  5. If NO candidate is safe (e.g. the plant is already outside the envelope),

     it falls back to the move that best restores the envelope -- backing the

     choke off to raise the pressures.



Because a first-order response moves monotonically from the current output to

its steady state, guaranteeing steady-state feasibility (plus a margin sized to

the process noise) is enough to guarantee the whole trajectory stays inside the

envelope. That is what makes this controller safe-by-construction: it never
```

```
commands a move it has not proven safe, and it knows when to say no.

"""

from __future__ import annotations



from dataclasses import dataclass, field



import numpy as np



from dataio import OUTPUTS

from envelope import PRESSURE_OUTPUTS, DEFAULT_ENVELOPE, Envelope

from model_id import ARXModel





@dataclass

class ControlDecision:

    """Diagnostics for a single control step (for logging / plotting)."""



    choke: float

    feasible: bool           # was at least one candidate safe?

    reached_target: bool     # is the chosen steady-state Q within tol of target?

    predicted_Q_ss: float    # steady-state Q the chosen choke settles to

    n_candidates: int

    n_feasible: int





@dataclass
```

```
class ChokeMPC:

    """Constrained predictive choke controller.



    Parameters

    ----------

    models : identified ARX(1) model per output.

    envelope : operating envelope (constraints). Defaults to DEFAULT_ENVELOPE.

    horizon : prediction horizon in steps for the safety check.

    grid_step : choke resolution (%) for the candidate search.

    move_penalty : lambda weighting on (delta_choke)^2 in the cost.

    safety_sigma : safety margin is safety_sigma * process-noise-std per pressure.

    safety_floor : minimum safety margin (psi) regardless of noise.

    target_tol : |Q_ss - target| below this counts as "target reached".

    """



    models: dict[str, ARXModel]

    envelope: Envelope = DEFAULT_ENVELOPE

    horizon: int = 15

    grid_step: float = 0.5

    move_penalty: float = 1e-3

    safety_sigma: float = 4.0

    safety_floor: float = 2.0

    target_tol: float = 1.0

    last_info: ControlDecision | None = field(default=None, init=False)



    def __post_init__(self) -> None:
```

```
        missing = [o for o in OUTPUTS if o not in self.models]
        if missing:
            raise ValueError(f"ChokeMPC missing models for outputs: {missing}")
        # Per-pressure safety margins, sized to the fitted process noise but
        # never below the floor.
        self.margins: dict[str, float] = {
            p: max(self.safety_floor, self.safety_sigma * self.models[p].resid_std)
            for p in PRESSURE_OUTPUTS
        }

    # -- internals ------------------------------------------------------------
    def _candidates(self, u_current: float) -> np.ndarray:
        """Ramp- and bound-feasible candidate choke positions, incl. current."""
        env = self.envelope
        lo = env.clip_choke(u_current - env.max_ramp)
        hi = env.clip_choke(u_current + env.max_ramp)
        grid = np.arange(lo, hi + 1e-9, self.grid_step)
        grid = np.clip(grid, env.choke_min, env.choke_max)
        # Always include the exact current position (hold) as an option.
        grid = np.unique(np.append(grid, env.clip_choke(u_current)))
        return grid

    def _is_safe(self, u_cand: float, state: dict[str, float]) -> bool:
        """True if holding u_cand keeps every pressure above its bound+margin.

        Checks the predicted transient over the horizon and the steady state.
```

```
        """
        mins = self.envelope.pressure_mins()
        for p in PRESSURE_OUTPUTS:
            m = self.models[p]
            limit = mins[p] + self.margins[p]
            # Steady state is the eventual (and, for a monotonic first-order
            # move, the worst) value when opening the choke.
            if m.steady_state(u_cand) < limit:
                return False
            # Transient trajectory from the current measured state.
            y = state[p]
            for _ in range(self.horizon):
                y = m.predict_next(y, u_cand)
                if y < limit:
                    return False
        return True


    # -- public API ---------------------------------------------------------
    def decide(
        self,
        q: float,
        whp: float,
        flp: float,
        bhp: float,
        u_current: float,
        target: float,
```

```
    ) -> float:
        """Return the next choke position (%) given the current measurements."""
        state = {"Q": q, "WHP": whp, "FLP": flp, "BHP": bhp}
        candidates = self._candidates(u_current)

        q_model = self.models["Q"]
        feasible: list[tuple[float, float, float]] = []  # (cost, u, Q_ss)
        for u_cand in candidates:
            if not self._is_safe(u_cand, state):
                continue
            q_ss = q_model.steady_state(u_cand)
            cost = (q_ss - target) ** 2 + self.move_penalty * (u_cand - u_current) ** 2
            feasible.append((cost, float(u_cand), float(q_ss)))

        if feasible:
            cost, u_next, q_ss = min(feasible, key=lambda t: t[0])
            reached = abs(q_ss - target) <= self.target_tol
            self.last_info = ControlDecision(
                choke=u_next,
                feasible=True,
                reached_target=reached,
                predicted_Q_ss=q_ss,
                n_candidates=len(candidates),
                n_feasible=len(feasible),
            )
            return u_next
```

```
        # No safe candidate: back off toward the move that best restores the
        # envelope (maximise the worst steady-state pressure margin).
        def recovery_margin(u_cand: float) -> float:
            return min(
                self.models[p].steady_state(u_cand) - self.envelope.pressure_mins()[p]
                for p in PRESSURE_OUTPUTS
            )

        u_next = float(max(candidates, key=recovery_margin))
        self.last_info = ControlDecision(
            choke=u_next,
            feasible=False,
            reached_target=False,
            predicted_Q_ss=q_model.steady_state(u_next),
            n_candidates=len(candidates),
            n_feasible=0,
        )
        return u_next


@dataclass
class NaiveController:
    """Baseline: proportional Q-tracking with NO envelope awareness.

    Respects only the physical choke bounds and the ramp limit -- it chases the
```

```
    oil-rate target regardless of pressure. Used to quantify how many envelope
    violations the safety-first MPC prevents.
    """

    envelope: Envelope = DEFAULT_ENVELOPE
    gain: float = 0.5  # % choke per (bbl/hr) of oil-rate error

    def decide(
        self,
        q: float,
        whp: float,
        flp: float,
        bhp: float,
        u_current: float,
        target: float,
    ) -> float:
        """Return the next choke position, ignoring the pressure envelope."""
        delta = self.gain * (target - q)
        return self.envelope.apply_ramp(u_current + delta, u_current)


if __name__ == "__main__":
    from model_id import identify_all
    from dataio import load_step_test
    from simulator import STARTUP_CHOKE, STARTUP_STATE, WellSimulator
```

```
mdls = identify_all(load_step_test())

ctrl = ChokeMPC(mdls)

print("Safety margins (psi):", {k: round(v, 2) for k, v in ctrl.margins.items()})



sim = WellSimulator(mdls, u0=STARTUP_CHOKE, y0=STARTUP_STATE, noise=True, seed=1)

u = STARTUP_CHOKE

q, whp, flp, bhp = sim.measure()

print("\nInfeasible-target demo (target=185, max safe ~156):")

for k in range(40):

    u = ctrl.decide(q, whp, flp, bhp, u, target=185.0)

    q, whp, flp, bhp = sim.step(u)

info = ctrl.last_info

print(f"  settled choke={u:.1f}%  Q={q:.1f}  WHP={whp:.1f}  feasible={info.feasible}")
```

plotting.py

```
"""Plotting: the required 6-trend scenario figure and the model-validation plot.



All figures are rendered with the non-interactive Agg backend so the pipeline

runs headless. Colours and layout are kept deliberately plain and legible for a

process-control audience.

"""

from __future__ import annotations



from pathlib import Path

from typing import Any



import matplotlib



matplotlib.use("Agg")

import matplotlib.pyplot as plt  # noqa: E402

import numpy as np  # noqa: E402



from dataio import OUTPUTS, to_canonical  # noqa: E402

from envelope import Envelope  # noqa: E402

from model_id import ARXModel, free_run  # noqa: E402



# Axis labels for each output.

_YLABEL = {

    "Q": "Oil rate\n(bbl/hr)",

    "WHP": "WHP (psi)",
```

```
    "FLP": "FLP (psi)",
    "BHP": "BHP (psi)",
}



def plot_model_validation(
    models: dict[str, ARXModel], df: Any, path: str | Path
) -> Path:
    """Overlay each identified model's open-loop free-run on the measured data."""
    canon = df if "u" in df.columns else to_canonical(df)
    t = canon["t"].to_numpy(float)
    u = canon["u"].to_numpy(float)

    fig, axes = plt.subplots(2, 2, figsize=(11, 7), sharex=True)
    fig.suptitle(
        "Model identification: ARX(1) free-run vs. step-test data", fontsize=13
    )

    for ax, name in zip(axes.flat, OUTPUTS):
        y_meas = canon[name].to_numpy(float)
        y_sim = free_run(models[name], u, y0=y_meas[0])
        ax.plot(t, y_meas, color="0.35", lw=1.2, label="data")
        ax.plot(t, y_sim, color="tab:red", lw=1.6, ls="--", label="ARX(1) model")
        m = models[name]
        ax.set_title(
            f"{name}:  gain={m.gain:+.2f},  tau={m.tau_hr:.1f} hr", fontsize=10
```

```
        )
        ax.set_ylabel(_YLABEL[name])
        ax.grid(True, alpha=0.3)
        ax.legend(fontsize=8, loc="best")

    for ax in axes[-1]:
        ax.set_xlabel("Time (hr)")

    fig.tight_layout(rect=(0, 0, 1, 0.96))
    path = Path(path)
    fig.savefig(path, dpi=130)
    plt.close(fig)
    return path


def plot_scenario(result: Any, envelope: Envelope, path: str | Path) -> Path:
    """Render the required 6-trend figure for one closed-loop scenario.

    The six trends are: Target Oil Rate, Actual Oil Rate, WHP, FLP, BHP and
    Choke Position. Oil-rate target and actual share the top panel; each
    pressure panel shows its lower-bound constraint; the choke panel shows the
    ramp context.

    ``result`` is any object exposing arrays t, target, Q, WHP, FLP, BHP, u and
    a string ``label`` / ``name`` (duck-typed to avoid a circular import).
    """
```

```
    t = np.asarray(result.t, float)



    fig, axes = plt.subplots(5, 1, figsize=(10, 11), sharex=True)

    title = getattr(result, "title", getattr(result, "name", "Scenario"))

    fig.suptitle(title, fontsize=13)



    # Panel 1: oil rate -- target + actual (two of the six trends).

    ax = axes[0]

    ax.plot(t, result.target, color="tab:blue", lw=1.8, ls="--", label="Target oil rate")

    ax.plot(t, result.Q, color="tab:green", lw=1.6, label="Actual oil rate")

    ax.set_ylabel(_YLABEL["Q"])

    ax.grid(True, alpha=0.3)

    ax.legend(fontsize=8, loc="best")



    # Panels 2-4: pressures with their lower-bound constraints.

    mins = envelope.pressure_mins()

    colors = {"WHP": "tab:red", "FLP": "tab:orange", "BHP": "tab:purple"}

    for ax, name in zip(axes[1:4], ("WHP", "FLP", "BHP")):

        series = np.asarray(getattr(result, name), float)

        ax.plot(t, series, color=colors[name], lw=1.5, label=name)

        ax.axhline(mins[name], color="k", ls=":", lw=1.3, label=f"{name}_min = {mins[name]:g}")

        # Shade any samples that fall below the bound (should be none for MPC).

        breach = series < mins[name]

        if breach.any():

            ax.fill_between(t, mins[name], series, where=breach, color="red", alpha=0.3)

        ax.set_ylabel(_YLABEL[name])
```

```
        ax.grid(True, alpha=0.3)
        ax.legend(fontsize=8, loc="best")

    # Panel 5: choke position.
    ax = axes[4]
    ax.plot(t, result.u, color="tab:brown", lw=1.6, label="Choke position")
    ax.set_ylabel("Choke (%)")
    ax.set_ylim(0, 100)
    ax.grid(True, alpha=0.3)
    ax.legend(fontsize=8, loc="best")
    ax.set_xlabel("Time (hr)")

    fig.tight_layout(rect=(0, 0, 1, 0.97))
    path = Path(path)
    fig.savefig(path, dpi=130)
    plt.close(fig)
    return path


def plot_baseline_comparison(
    mpc_result: Any, naive_result: Any, envelope: Envelope, path: str | Path
) -> Path:
    """Compare safety-first MPC vs the naive baseline on the same scenario.

    Highlights how the naive controller breaches the pressure envelope while the
    MPC stays inside it, at the cost of chasing an infeasible oil-rate target.
```

```
    """
    t = np.asarray(mpc_result.t, float)
    mins = envelope.pressure_mins()

    fig, axes = plt.subplots(3, 1, figsize=(10, 9), sharex=True)
    fig.suptitle(
        f"Safety-first MPC vs. naive baseline -- {getattr(mpc_result, 'title', '')}",
        fontsize=12,
    )

    ax = axes[0]
    ax.plot(t, mpc_result.target, color="tab:blue", ls="--", lw=1.6, label="Target")
    ax.plot(t, mpc_result.Q, color="tab:green", lw=1.6, label="MPC oil rate")
    ax.plot(t, naive_result.Q, color="tab:gray", lw=1.4, label="Naive oil rate")
    ax.set_ylabel(_YLABEL["Q"])
    ax.grid(True, alpha=0.3)
    ax.legend(fontsize=8)

    ax = axes[1]
    ax.plot(t, mpc_result.WHP, color="tab:green", lw=1.6, label="MPC WHP")
    ax.plot(t, naive_result.WHP, color="tab:gray", lw=1.4, label="Naive WHP")
    ax.axhline(mins["WHP"], color="k", ls=":", lw=1.3, label=f"WHP_min = {mins['WHP']:g}")
    naive_whp = np.asarray(naive_result.WHP, float)
    breach = naive_whp < mins["WHP"]
    if breach.any():
        ax.fill_between(t, mins["WHP"], naive_whp, where=breach, color="red", alpha=0.3,
```

```
                      label="Naive breach")
    ax.set_ylabel(_YLABEL["WHP"])
    ax.grid(True, alpha=0.3)
    ax.legend(fontsize=8)

    ax = axes[2]
    ax.plot(t, mpc_result.u, color="tab:green", lw=1.6, label="MPC choke")
    ax.plot(t, naive_result.u, color="tab:gray", lw=1.4, label="Naive choke")
    ax.set_ylabel("Choke (%)")
    ax.set_ylim(0, 100)
    ax.grid(True, alpha=0.3)
    ax.legend(fontsize=8)
    ax.set_xlabel("Time (hr)")

    fig.tight_layout(rect=(0, 0, 1, 0.96))
    path = Path(path)
    fig.savefig(path, dpi=130)
    plt.close(fig)
    return path
```

scenarios.py

```
"""Closed-loop scenario runners, metrics, and the naive-baseline comparison.



Three demonstration scenarios from the brief:



  A - Startup -> Target: bring the well from startup to a fixed production target.

  B - Target Tracking:   target steps mid-run (100 -> 150 bbl/hr).

  C - Infeasible Target: request exceeds what is safe -> settle at the max safe rate.



Each scenario is run closed-loop against the WellSimulator and summarised with

metrics (constraint violations, tracking error, settled rate). The same

scenarios are also run with a naive, envelope-unaware baseline to quantify how

many violations the safety-first MPC prevents.

"""

from __future__ import annotations



from dataclasses import dataclass, field

from typing import Callable



import numpy as np



from controller import ChokeMPC, NaiveController

from dataio import OUTPUTS

from envelope import DEFAULT_ENVELOPE, PRESSURE_OUTPUTS, Envelope

from model_id import ARXModel

from simulator import STARTUP_CHOKE, STARTUP_STATE, WellSimulator
```

```
@dataclass

class ScenarioResult:

    """Time series and metrics for one closed-loop run."""



    name: str

    title: str

    t: np.ndarray

    target: np.ndarray

    Q: np.ndarray

    WHP: np.ndarray

    FLP: np.ndarray

    BHP: np.ndarray

    u: np.ndarray

    metrics: dict = field(default_factory=dict)




@dataclass

class ScenarioSpec:

    """Definition of a scenario: how long, what target, where it starts."""



    name: str

    title: str

    n_steps: int

    target_fn: Callable[[int], float]
```

```
    u0: float = STARTUP_CHOKE
    y0: dict[str, float] | None = None
    feasible: bool = True  # is the (final) target physically achievable safely?

    def target_series(self) -> np.ndarray:
        return np.array([self.target_fn(k) for k in range(self.n_steps)], dtype=float)


def run_closed_loop(
    controller,
    models: dict[str, ARXModel],
    spec: ScenarioSpec,
    envelope: Envelope = DEFAULT_ENVELOPE,
    noise: bool = True,
    seed: int = 0,
) -> ScenarioResult:
    """Run one scenario closed-loop and return the recorded time series.

    Causal loop: at each step we record the current measurement and the choke
    that produced it, ask the controller for the next choke, then advance the
    plant one interval.
    """
    targets = spec.target_series()
    n = spec.n_steps
    y0 = spec.y0 if spec.y0 is not None else STARTUP_STATE
    sim = WellSimulator(models, u0=spec.u0, y0=y0, noise=noise, seed=seed)
```

```
    t = np.arange(n, dtype=float) * envelope.Ts_hr
    Q = np.empty(n); WHP = np.empty(n); FLP = np.empty(n); BHP = np.empty(n)
    u_rec = np.empty(n)

    u = float(spec.u0)
    q, whp, flp, bhp = sim.measure()
    for k in range(n):
        Q[k], WHP[k], FLP[k], BHP[k], u_rec[k] = q, whp, flp, bhp, u
        u = controller.decide(q, whp, flp, bhp, u, targets[k])
        q, whp, flp, bhp = sim.step(u)

    result = ScenarioResult(
        name=spec.name, title=spec.title, t=t, target=targets,
        Q=Q, WHP=WHP, FLP=FLP, BHP=BHP, u=u_rec,
    )
    result.metrics = compute_metrics(result, envelope, spec)
    return result


def compute_metrics(
    result: ScenarioResult, envelope: Envelope, spec: ScenarioSpec
) -> dict:
    """Summarise a run: violations, tracking error, settled rate."""
    mins = envelope.pressure_mins()
    series = {"WHP": result.WHP, "FLP": result.FLP, "BHP": result.BHP}
```

```
violations_by_pressure = {
    p: int(np.sum(series[p] < mins[p])) for p in PRESSURE_OUTPUTS
}
violations_total = int(sum(violations_by_pressure.values()))

min_pressures = {p: float(np.min(series[p])) for p in PRESSURE_OUTPUTS}
worst_margin = float(
    min(min_pressures[p] - mins[p] for p in PRESSURE_OUTPUTS)
)

# Tracking assessed over the last 10 steps (settled window).
tail = slice(-10, None)
final_target = float(result.target[-1])
final_Q = float(np.mean(result.Q[tail]))
steady_state_error = final_Q - final_target
max_abs_tracking_error = float(np.max(np.abs(result.Q - result.target)))

return {
    "scenario": result.name,
    "target_feasible": spec.feasible,
    "final_target_bbl_hr": final_target,
    "settled_oil_rate_bbl_hr": round(final_Q, 2),
    "settled_choke_pct": round(float(np.mean(result.u[tail])), 2),
    "steady_state_error_bbl_hr": round(steady_state_error, 2),
    "max_abs_tracking_error_bbl_hr": round(max_abs_tracking_error, 2),
```

```
        "violations_total": violations_total,
        "violations_by_pressure": violations_by_pressure,
        "min_pressures_psi": {p: round(v, 2) for p, v in min_pressures.items()},
        "worst_pressure_margin_psi": round(worst_margin, 2),
    }



# -- Scenario catalogue ---------------------------------------------------

def scenario_specs() -> list[ScenarioSpec]:
    """The three required demonstration scenarios (targets per HANDOFF Section 3)."""
    return [
        ScenarioSpec(
            name="A",
            title="Scenario A -- Startup to Target (130 bbl/hr)",
            n_steps=60,
            target_fn=lambda k: 130.0,
            feasible=True,
        ),
        ScenarioSpec(
            name="B",
            title="Scenario B -- Target Tracking (100 then 150 bbl/hr)",
            n_steps=100,
            target_fn=lambda k: 100.0 if k < 50 else 150.0,
            feasible=True,
        ),
        ScenarioSpec(
```

```
            name="C",
            title="Scenario C -- Infeasible Target (185 bbl/hr, settle at max safe)",
            n_steps=70,
            target_fn=lambda k: 185.0,
            feasible=False,
        ),
    ]


def run_all_mpc(
    models: dict[str, ARXModel],
    envelope: Envelope = DEFAULT_ENVELOPE,
    seed: int = 0,
) -> dict[str, ScenarioResult]:
    """Run all three scenarios with the safety-first MPC."""
    results: dict[str, ScenarioResult] = {}
    for spec in scenario_specs():
        ctrl = ChokeMPC(models, envelope=envelope)
        results[spec.name] = run_closed_loop(ctrl, models, spec, envelope, seed=seed)
    return results


def run_all_baseline(
    models: dict[str, ARXModel],
    envelope: Envelope = DEFAULT_ENVELOPE,
    seed: int = 0,
```

```
) -> dict[str, ScenarioResult]:

    """Run all three scenarios with the naive (envelope-unaware) baseline."""

    results: dict[str, ScenarioResult] = {}

    for spec in scenario_specs():

        ctrl = NaiveController(envelope=envelope)

        results[spec.name] = run_closed_loop(ctrl, models, spec, envelope, seed=seed)

    return results




if __name__ == "__main__":

    from dataio import load_step_test

    from model_id import identify_all



    mdls = identify_all(load_step_test())

    mpc = run_all_mpc(mdls)

    naive = run_all_baseline(mdls)

    print(f"{'scen':<5}{'MPC viol':>10}{'naive viol':>12}{'settled Q':>12}{'ss err':>9}")

    for name in ("A", "B", "C"):

        m = mpc[name].metrics

        b = naive[name].metrics

        print(

            f"{name:<5}{m['violations_total']:>10}{b['violations_total']:>12}"

            f"{m['settled_oil_rate_bbl_hr']:>12}{m['steady_state_error_bbl_hr']:>9}"

        )
```

main.py

```
"""End-to-end pipeline for the Autonomous Production Choke Controller (Problem 3A).



Running ``python main.py`` will:



  1. Load and validate the step-test dataset.

  2. Identify the ARX(1) dynamic models (prints gains + time constants) and save

     a model-validation plot.

  3. Run the three demonstration scenarios (A, B, C) closed-loop with the

     safety-first MPC, saving the required 6-trend plot for each.

  4. Run the naive baseline for comparison and save a baseline-vs-MPC plot.

  5. Write results/metrics.json.

  6. ASSERT zero envelope violations across A, B and C (fails loudly otherwise).



All artefacts land in ``results/`` and are kept small for the 10 MB submission

limit.

"""

from __future__ import annotations



import json

from pathlib import Path



from dataio import infer_sample_time, load_step_test

from envelope import DEFAULT_ENVELOPE

from model_id import format_report, identify_all, validation_fit

from plotting import plot_baseline_comparison, plot_model_validation, plot_scenario
```

```
from scenarios import run_all_baseline, run_all_mpc



HERE = Path(__file__).resolve().parent

RESULTS = HERE / "results"




def main() -> int:

    RESULTS.mkdir(exist_ok=True)

    env = DEFAULT_ENVELOPE



    # 1. Data ------------------------------------------------------------

    print("=" * 70)

    print("Autonomous Production Choke Controller -- Problem 3A")

    print("=" * 70)

    df = load_step_test()

    ts = infer_sample_time(df)

    print(f"Loaded step-test: {len(df)} rows, Ts = {ts} hr\n")



    # 2. Model identification --------------------------------------------

    models = identify_all(df, Ts=ts)

    print(format_report(models))

    print("\nFree-run model validation (open-loop vs data):")

    model_val = {}

    for name, m in models.items():

        v = validation_fit(m, df)

        model_val[name] = {k: round(val, 3) for k, val in v.items()}
```

```
    print(f"  {name:<4} RMSE={v['rmse']:.2f} ({v['rmse_pct_of_span']:.1f}% of span)")

val_plot = plot_model_validation(models, df, RESULTS / "model_validation.png")

print(f"  -> {val_plot.relative_to(HERE)}")



# Report the envelope consequence for context.

whp = models["WHP"]

q = models["Q"]

u_whp_bind = (whp.b0 - env.WHP_min) / (-whp.b1)

max_safe_rate = q.steady_state(u_whp_bind)

print(

    f"\nEnvelope: WHP hits {env.WHP_min} psi at choke {u_whp_bind:.1f}% "

    f"-> hard max safe rate ~{max_safe_rate:.0f} bbl/hr (WHP binds first)."

)



# 3. Closed-loop scenarios (safety-first MPC) -----------------------

print("\n" + "-" * 70)

print("Closed-loop scenarios (safety-first MPC):")

mpc = run_all_mpc(models, envelope=env)

for name in ("A", "B", "C"):

    res = mpc[name]

    p = plot_scenario(res, env, RESULTS / f"scenario_{name}.png")

    m = res.metrics

    print(

        f"  {name}: violations={m['violations_total']}, "

        f"settled Q={m['settled_oil_rate_bbl_hr']} "

        f"(target {m['final_target_bbl_hr']:.0f}), "
```

```
        f"worst margin={m['worst_pressure_margin_psi']} psi  -> {p.name}"
    )


# 4. Naive baseline for comparison ---------------------------------
print("\nNaive baseline (envelope-unaware) for comparison:")
naive = run_all_baseline(models, envelope=env)
for name in ("A", "B", "C"):
    print(f"  {name}: violations={naive[name].metrics['violations_total']}")
cmp_plot = plot_baseline_comparison(
    mpc["C"], naive["C"], env, RESULTS / "baseline_comparison_C.png"
)
print(f"  -> {cmp_plot.name}")


# 5. Metrics -------------------------------------------------------
metrics = {
    "problem": "3A -- Autonomous Production Choke Controller",
    "sample_time_hr": ts,
    "envelope": {
        "WHP_min": env.WHP_min, "FLP_min": env.FLP_min, "BHP_min": env.BHP_min,
        "choke_min": env.choke_min, "choke_max": env.choke_max,
        "max_ramp_pct_per_step": env.max_ramp,
    },
    "models": {
        name: {
            "a": round(m.a, 4), "gain_b1": round(m.b1, 4), "b0": round(m.b0, 3),
            "tau_hr": round(m.tau_hr, 2), "noise_std": round(m.resid_std, 3),
```

```
            }
            for name, m in models.items()
        },
        "model_validation": model_val,
        "hard_max_safe_rate_bbl_hr": round(float(max_safe_rate), 1),
        "scenarios_mpc": {name: mpc[name].metrics for name in ("A", "B", "C")},
        "scenarios_baseline": {
            name: {
                "violations_total": naive[name].metrics["violations_total"],
                "violations_by_pressure": naive[name].metrics["violations_by_pressure"],
                "settled_oil_rate_bbl_hr": naive[name].metrics["settled_oil_rate_bbl_hr"],
            }
            for name in ("A", "B", "C")
        },
    }
    metrics_path = RESULTS / "metrics.json"
    metrics_path.write_text(json.dumps(metrics, indent=2))
    print(f"\nWrote {metrics_path.relative_to(HERE)}")



    # 6. Hard verification ------------------------------------------------
    total_violations = sum(mpc[n].metrics["violations_total"] for n in ("A", "B", "C"))
    baseline_violations = sum(naive[n].metrics["violations_total"] for n in ("A", "B", "C"))
    print("\n" + "=" * 70)
    print(f"MPC total envelope violations across A/B/C : {total_violations}")
    print(f"Naive baseline total violations across A/B/C: {baseline_violations}")
    print("=" * 70)
```

```
    assert total_violations == 0, (
        f"SAFETY FAILURE: MPC produced {total_violations} envelope violations "
        "(expected 0)."
    )
    # A and B targets are feasible -> require good tracking.
    for name in ("A", "B"):
        err = abs(mpc[name].metrics["steady_state_error_bbl_hr"])
        assert err <= 2.0, f"Scenario {name} steady-state error {err} bbl/hr exceeds 2.0."
    print("VERIFIED: 0 violations across all scenarios; A/B track target within 2 bbl/hr.")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```